

Exploit:**

Exploit:

1. Explanation:-

WordPress utilizes WP_Query to resolve and serve content dynamically based on URL parameters. While GET parameters are sanitized and controlled via query_vars, the lesser-known facet is the overlap between GET, POST, and direct input superglobals.

Even when GET is blocked via query_vars, the POST payload is still subject to internal query resolution through parse_request()

2. Problem:-

Blocking page_id in GET introduces an orthogonal response profile, where GET calls return an explicit 403/404, but POST calls still return semantic leakage through subtle response variations.

3. How does this bypass Cloudflare:-

Most WAFs rely on GET-based rules (page_id in URL → blocked) but POST payload rules are weaker and nothing can be done about that.

4. Fix:

Wordpress:-

i)Add this to functions.php:-

```
add_action('init', function () {
    if (isset($_REQUEST['page_id'])) {
        unset($_REQUEST['page_id']); // Completely remove page_id
    }
});
```

ii)Override WP_Query to prevent direct access:-

```
add_action('pre_get_posts', function ($query) {
    if (!is_admin() && isset($_REQUEST['page_id'])) {
        $query->set('page_id', 0); // Nullify the request
    }
});
```

iii)Add this to .htaccess:-

```
Header set Content-Security-Policy "default-src 'none'; script-src 'self'; style-src 'self'; frame-ancestors 'none';"
```

iv) Prevent attacks using other requests (Add to .htaccess):-

```
<LimitExcept GET POST>  
    Deny from all  
</LimitExcept>
```

Cloudflare:-

“Block or challenge” this cloudflare firewall rule:-

```
(http.request.method eq "POST" and http.request.body contains "page_id=")
```